COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/28/2026	**DATE SUBMITTED:**	09/2/2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **Prelim Examination**

1. PROCEDURE OUTPUT

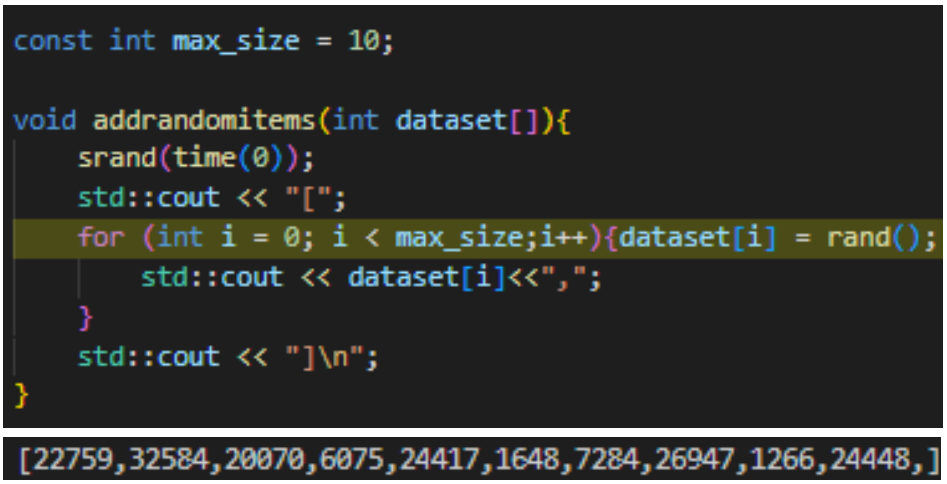
Screenshot	
Observations	The array creation method consists of setting a constant max_size which is implemented in the generation of random numbers that fills up the max size of the array. The random generated numbers are automatically printed by the cout function and enclosed in a bracket.

Table 6-1. Data Generated and Observations.

Code

```cpp
#include <iostream>
#include <cstdlib>//random gen
#include <time.h>//seed

const int max_size = 10;

void addrandomitems(int dataset[]){
    srand(time(0));
    std::cout << "[";
    for (int i = 0; i < max_size;i++){dataset[i] = rand();
        std::cout << dataset[i]<<",";
    }
    std::cout << "]\n";
}

int main(){

    int dataset[max_size] = {1,2,3,4,5,6,7,8,9,10};
    addrandomitems(dataset);

    int itemToFind;
    std::cout << "Enter data to find:\n";
    std::cin >> itemToFind;

    return 0;
}
```

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//---------------------- ARRAY -------------------------

template<typename T>
void linearSearch(int size,T item,T array[]){
    for (int i = 0; i < size;i++){
        if(array[i] == item){
            std::cout << "Data Found at index " << i << "\n";
            return;
        }
    }
    std::cout << "Data not found\n";
}
```

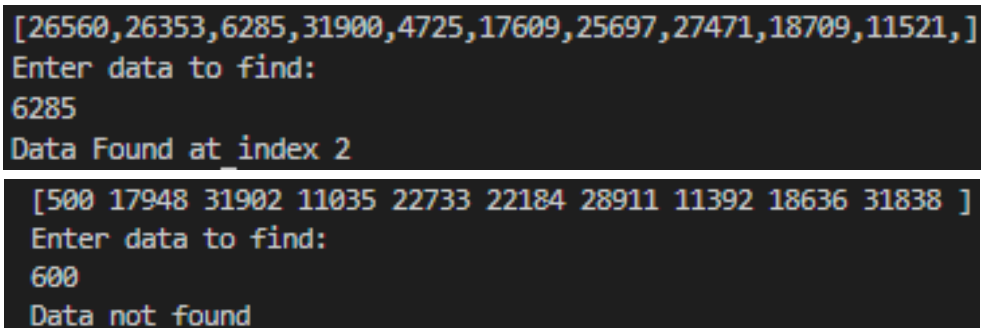
Output	
Observations	The random generated array elements displayed in the terminal are used to conduct the linear search. The linear search iterates the array elements until it matches the user input which is called the item. After that, it displays the index of the item in the array. If the item is not in the array element, it prints data not found.

Table 6-2a. Linear Search for Arrays

Code

```
#include <iostream>
#include <cstdlib>//random gen
#include <time.h>//seed

#include "node.h"
#include "searchingAlgo.h"

const int max_size = 10;

void addRandItems(int dataset[]){
    srand(time(0));
    std::cout << "[";
    for (int i = 0; i < max_size;i++){    dataset[i] = rand();
        std::cout << dataset[i]<<",";
    }
    std::cout << "]\n";
}

int main(){

    int dataset[max_size] = {1,2,3,4,5,6,7,8,9,10};
    addRandItems(dataset);

    int itemToFind;
    std::cout << "Enter data to find:\n";
    std::cin >> itemToFind;

    linearSearch(max_size,itemToFind,dataset);

    Node<int> *num1  = newNode(10);
    Node<int> *num2  = newNode(20);
    Node<int> *num3  = newNode(30);
    Node<int> *num4  = newNode(40);
    Node<int> *num5  = newNode(50);
    Node<int> *num6  = newNode(60);
    Node<int> *num7  = newNode(70);
    Node<int> *num8  = newNode(80);
    Node<int> *num9  = newNode(90);
    Node<int> *num10 = newNode(100);

    num1->next  = num2;
    num2->next  = num3;
    num3->next  = num4;
    num4->next  = num5;
    num5->next  = num6;
    num6->next  = num7;
    num7->next  = num8;
    num8->next  = num9;
    num9->next  = num10;

    return 0;
}
```

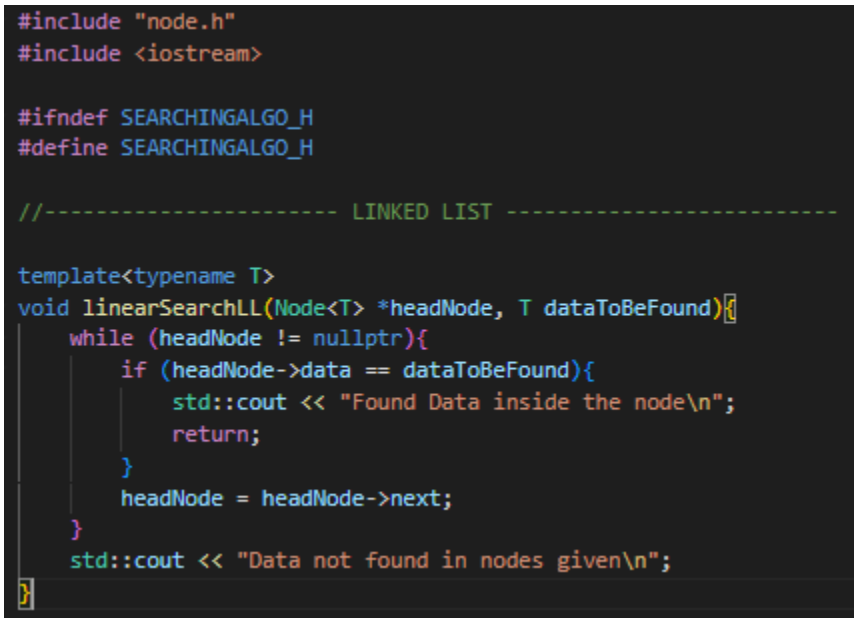
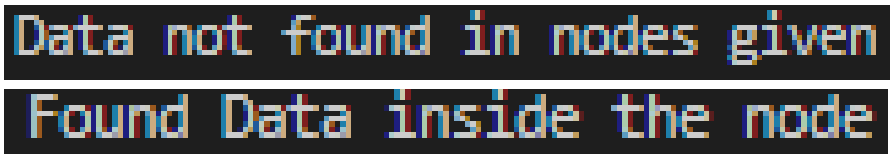
	
Output	
Observations	The iteration happens linearly, from the start until it finds the data. If the data is not in the does, then it will print data not found. The runtime execution is slower for larger scale operations.

Table 6-2b. Linear Search for Linked List

Code

```cpp
#include <iostream>
#include <cstdlib>//random gen
#include <time.h>//seed

//#include "node.h"
#include "searchingAlgo.h"

const int max_size = 10;

void addrandomitems(int dataset[]){
    srand(time(0));
    std::cout << "[";
    for (int i = 0; i < max_size;i++){dataset[i] = rand();
        std::cout << dataset[i]<<" ";
    }
    std::cout << "]\n";
}

int main(){

    int dataset[max_size] = {1,2,3,4,5,6,7,8,9,10};
    addrandomitems(dataset);

    int itemToFind;
    std::cout << "Enter data to find:\n";
    std::cin >> itemToFind;

    binarySearchArr(dataset, max_size, itemToFind);

    return 0;
}
```

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//---------------------- ARRAY -------------------------

template<typename T>
void binarySearchArr(T data[],int arrSize,T dataFind){
    int low = 0;
    int high = arrSize - 1;
    int middle;

    while (low <= high){
        middle = (int)((low+high)/2);
        if (dataFind == data[middle]){
            std::cout << "The data was found\n";
            return;
        }
        else if (dataFind < data[middle]){
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    std::cout << "The data was not found\n";
}
```

Output

```
[1519 29115 2195 14917 29616 2140 16956 15587 25676 14185 ]
Enter data to find:
2916
The data was not found
```

```
[1737 28362 19447 21527 32565 3087 28882 16067 17840 28248 ]
Enter data to find:
1737
The data was found
```

Observations

The runtime execution of binary search is much faster than the linear search. It also has many conditions when programmed. The binary search indicates lower bound and upper bounds of searching. If the data is in between the upper and lower bound, then the program will print successfully. If it isn't, it will change the upper or lower bound to find the element. If not in the array, it will print not found.

Table 6-3a. Binary Search for Arrays

Code

```
#include <iostream>
#include <cstdlib>//random gen
#include <time.h>//seed

#include "node.h"
#include "searchingAlgo.h"

int main(){

    int itemToFind;

    Node<int> *num1  = newNode(10);
    Node<int> *num2  = newNode(20);
    Node<int> *num3  = newNode(30);
    Node<int> *num4  = newNode(40);
    Node<int> *num5  = newNode(50);
    Node<int> *num6  = newNode(60);
    Node<int> *num7  = newNode(70);
    Node<int> *num8  = newNode(80);
    Node<int> *num9  = newNode(90);
    Node<int> *num10 = newNode(100);

    num1->next  = num2;
    num2->next  = num3;
    num3->next  = num4;
    num4->next  = num5;
    num5->next  = num6;
    num6->next  = num7;
    num7->next  = num8;
    num8->next  = num9;
    num9->next  = num10;

    binarySearchLL(num1, 60);

    delNodes(&num1);

    return 0;
}
```

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//---------------------- LINKED LIST -------------------------
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind) {
    Node<T>* last = nullptr;

    while (start != last) {
        Node<T>* middle = getMiddle(start, last);

        if (middle->data == dataFind) {
            std::cout << "Data found\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;
        }
        else {
            start = middle->next;
        }
    }

    std::cout << "Data not found\n";
}

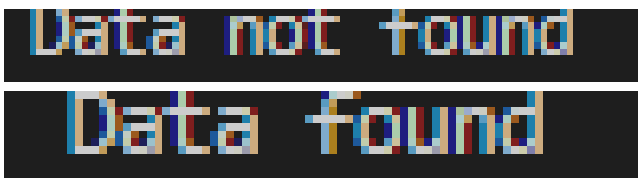
template<typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* end){
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != end && fast->next!= end){
        slow = slow->next;

        fast = fast->next->next;
    }

    return slow;
}
```

Output



Observations	In a binary search linked list implementation, the two nodes, fast and slow both start at the front then the fast version does two nodes at each iteration, while the slow node does one. Once the fast one reaches the end, the slow one is in the middle. If the object still isn't found by then, they will shrink the boundary of the searching area and repeat the same process until it finds the correct target. Binary search has a faster run time than linear search.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:
To find the key '18' in the array and linked list implementation of sequential search, the program will have to execute a comparison twice since the nearest appearance of the character '18' is in the first index and the second node or the first node index.

Problem 2:

```
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//---------------------- LINKED LIST -------------------------

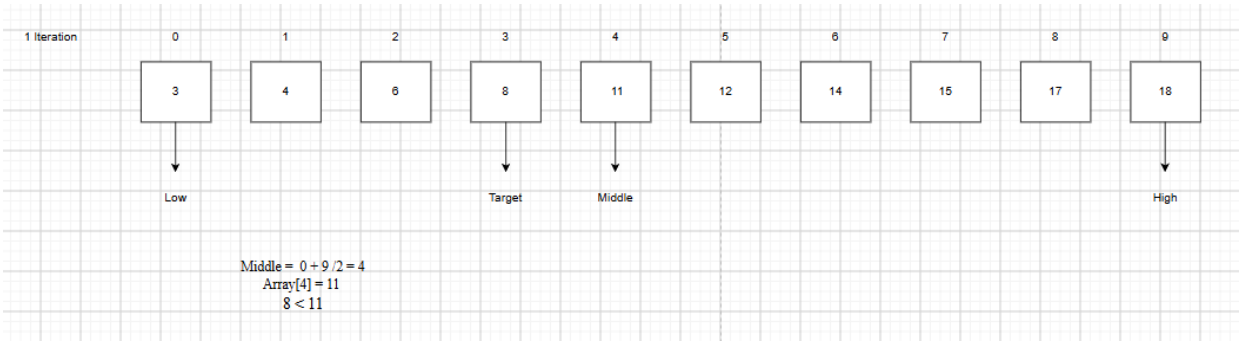
template<typename T>
void linearSearchLL(Node<T> *headNode, T dataToBeFound){
    int count = 0;
    while (headNode != nullptr){
        if (headNode->data == dataToBeFound){
            count++;
        }
        headNode = headNode->next;
    }
    if (count > 0){
        std::cout << dataToBeFound << " appears " << count << " number of times." << std::endl;
    }
    else {
        std::cout << "data not found!" << std::endl;
    }
}

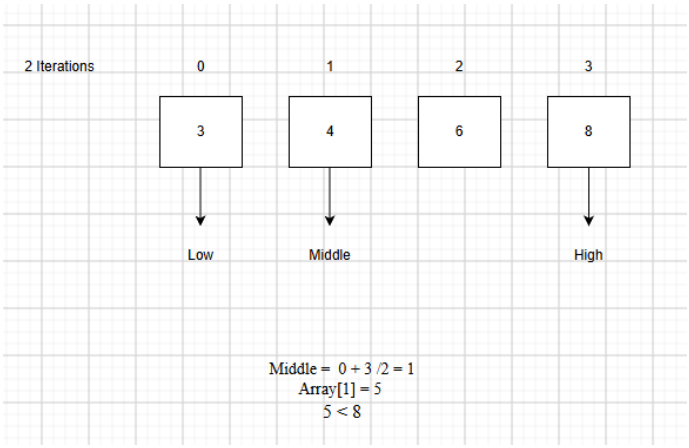
//---------------------- ARRAY -------------------------

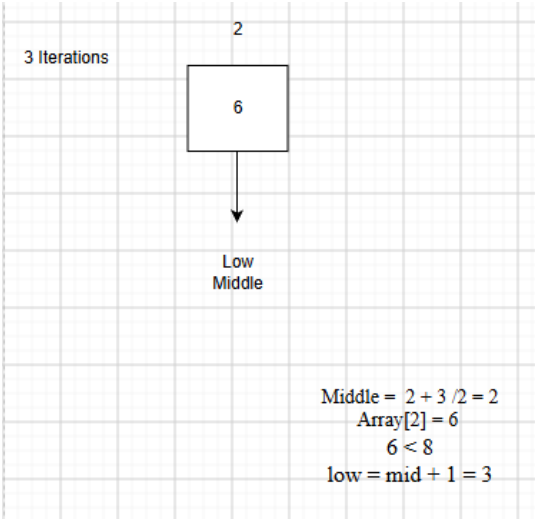
template<typename T>
void linearSearch(int size,T item,T array[]){
    int count = 0;
    for (int i = 0; i < size;i++){
        if(array[i] == item){
            count++;
        }
    }
    if (count > 0){
        std::cout << item << " appears " << count << " number of times." << std::endl;
    }
    else {
        std::cout << "data not found!" << std::endl;
    }
}
```

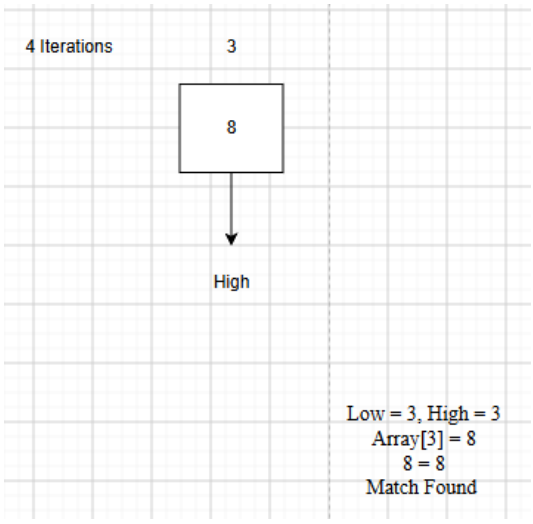
```
Enter data to find:
18
18 appears 2 number of times.
18 appears 2 number of times.
```

Problem 3:









```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//---------------------- ARRAY -------------------------

template<typename T>
void binarySearchArr(T data[], int arrSize, T dataFind){
    int low = 0;
    int high = arrSize - 1;
    int iteration = 1;

    while (low <= high){
        int mid = (low+high)/2;

        std::cout << "Iteration: " << iteration << ": low= " << low << ": high= " << high << ": middle= " << mid << ": value= " << data[mid] << "\n";

        if (dataFind == data[mid]){
            std::cout << "Match found at index: " << mid << std::endl;
            return;
        }
        else if (dataFind < data[mid]){
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
        iteration++;
    }
    std::cout << "Data was not found\n";
}
```

```
Iteration: 1: low= 0: high= 9: middle= 4: value= 11
Iteration: 2: low= 0: high= 3: middle= 1: value= 5
Iteration: 3: low= 2: high= 3: middle= 2: value= 6
Iteration: 4: low= 3: high= 3: middle= 3: value= 8
Match found at index: 3
```

Problem 4:

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

//---------------------- ARRAY -------------------------

template<typename T>
int RecBinarySearchArr(T data[], int low, int high, T dataFind, int iteration) {
    if (low > high) {
        std::cout << "Data not found\n";
        return -1;
    }

    int mid = (low + high) / 2;

    std::cout << "Iteration: " << iteration << ": low= " << low << ": high= " << high << ": middle= " << mid << ": value= " << data[mid] << "\n";

    if (dataFind == data[mid]) {
        std::cout << "Match found at index: " << mid << std::endl;
        return mid;
    }
    else if (dataFind < data[mid]) {
        return RecBinarySearchArr(data, low, mid - 1, dataFind, iteration + 1);
    }
    else {
        return RecBinarySearchArr(data, mid + 1, high, dataFind, iteration + 1);
    }
}

template<typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
    RecBinarySearchArr(data, 0, arrSize - 1, dataFind, 1);
}

#endif
```

```
Iteration: 1: low= 0: high= 9: middle= 4: value= 11
Iteration: 2: low= 0: high= 3: middle= 1: value= 5
Iteration: 3: low= 2: high= 3: middle= 2: value= 6
Iteration: 4: low= 3: high= 3: middle= 3: value= 8
Match found at index: 3
```

Conclusion:

I've learned the basic implementation of linked lists and arrays for sequential and binary search. I've also learned that faster searching methods depend on what type or how much data you have. Linear or sequential search is a type of searching that iterates sequentially, or in chronological manner. On the other hand, binary search is like a two way search which uses upper and lower boundaries to search for something. It also shrinks the data set in order to find the value that it looks for.

Basically, what we did in the supplementary activity are the concepts and applications of the two types or searching algorithms. The linear search usually uses for loops to find the item that it looks for in an array. In a linked list, it just uses one directional traversion from one node to the next node. Binary search in arrays usually uses while loops to find the target value. However, if certain conditions are met, such as if the index of low + high divided by 2 is not equal to the target value, then it shrinks the data set and repeats the same process until it finds the target. In a linked list implementation, a void function called the middle is used to indicate the middle and upper boundary. There are two nodes, the fast node which travels two nodes each iteration and the slow node which travels 1 node. If the fast node reaches the end, the slow node is in the middle. They are basically what shrinks the list to find the target value.

I think I did well in this activity, though I passed my activity late, I think that I really did a great job. However, seeing that I didn't have enough skills to pass it on time due to me being unable to cram it properly, I think that I still have a lot to learn especially since I'm just a beginner but overall, I think I did a great job as a beginner trying to do this activity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.